

Résumé exécutif

Forearm Lab est une PWA *mobile-first*, locale et optimisée, ciblant l'athlète amateur avancé (iPhone 13 Pro Max en priorité). Le but est de suivre des exercices de renforcement des avant-bras (répétitions, séries) de manière fluide, hors-ligne, avec sauvegarde locale et option d'export. Le produit visé sera minimal et premium : interface épurée, micro-interactions subtiles, touches de verre-morphisme bien dosées [46†L1-L4] [45†L233-L241] .

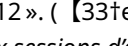
Le MVP couvre les fonctionnalités essentielles : écran d'accueil listant les exercices, écran d'exercice avec compteurs (répétitions/séries), suivi des performances (dates, progressions), et un menu d'export des données (JSON/CSV/SQLite). La PWA doit fonctionner *offline* de bout en bout [43†L119-L127]

[24†L109-L117] . L'architecture choisie s'appuie sur Next.js (App Router), TypeScript et React, avec Tailwind CSS pour l'UI. Le stockage local s'effectue via IndexedDB (par exemple Dexie.js) ou équivalent (p. ex. storage local JSON) [24†L109-L117] [43†L119-L127] . Le déploiement cible Vercel (complémentaire à Next.js).

Côté développement, nous adoptons une approche *agent-driven* : chaque domaine (produit, UX, dev front/back, sécurité) a un agent responsable autonome qui documente ses décisions. Les choix (design, technologie, sécurité) seront consignés dans un format standardisé (contexte, options, décision, justification). Chaque "agent" reporte périodiquement (, % complétion, niveau de risque) pour un suivi global transparent. Un pipeline CI/CD automatisé couvrira tests, audits et qualité (lint/TypeScript, tests unitaires, Lighthouse, scans de sécurité).

Sources clés : recommandations PWA offline [43†L119-L127] , design minimaliste/UX premium [48†L100-L109] [46†L1-L4] , bonnes pratiques *glassmorphism* [46†L1-L4] [45†L233-L241] , bases *offline-first* [24†L109-L117] [43†L75-L83] .

Vision produit & périmètre MVP

- **Vision:** Application épurée, mobile-first, dédiée au suivi d'exercices des avant-bras. Elle doit offrir une UX premium (interface claire, animations subtiles) tout en restant très légère et fonctionnant sans connexion internet.
- **Cible et usages:** Athlète amateur entraîné, adepte du home training, utilisera l'app en salle ou à domicile sur téléphone (priorité iPhone). L'UX mobile est prioritaire (iPhone 13 Pro Max spécifiquement) mais l'affichage desktop sera responsive et soigné.
- **Fonctionnalités MVP :**
 - **Liste d'exercices** (ex. « Flexions du poignet », « Extensions poignet »).
 - **Interface d'entraînement:** pour chaque exercice, affichage du nombre de séries et répétitions, minuterie et compteur. Exemple simplifié : bouton "+1 rép" ou minuterie d'une série, progression « Séries : 1/3, Rép : 5/12 ». ( Exemple d'écran d'accueil mobile, combinant suivi de calories et accès aux sessions d'entraînement).
 - **Suivi des performances:** journal local des séances (date, exercices, séries, répétitions totales). Visualisation simpliste (tableau ou graphique sommaire).
 - **Export/Import des données:** bouton « Exporter mes données » (CSV/JSON/SQLite), et possibilité de réimporter le fichier en cas de réinstallation.

- **Settings/PWA:** manifest.json, icône, splash screen pour installation, indicateur offline (icône ou bandeau).
- **Constraints & non-fonctionnel :** tout passe par le stockage local (IndexedDB, pas de serveur). Performance et réactivité maximales (pas de spinner perceptible **【24†L135-L143】**). Code internationalisé mais langue cible par défaut FR/EN (Anglais optionnel).

Personas et parcours utilisateur

1. **Persona principal:** *Julien, 30 ans, sportif amateur engagé.* Utilise son iPhone à la salle ou chez lui. A déjà un smartphone-friendly. Bien informé, attend des infos avancées (stats, astuces) sans entrave technique.
2. **Parcours mobile:**
3. **Lancement de l'app (mode offline):** il voit la liste des exercices ou « Aujourd'hui », peut cliquer sur un exercice. (Écran d'accueil simple, sans chargement à cause du cache PWA) **【43†L119-L127】** **【24†L135-L143】** .
4. **Entraînement:** dans l'écran « Flexions du poignet », Julien appuie sur un gros bouton « +1 » ou sur un timer qui compte les répétitions/séries. L'interface lui donne un feedback animé (micro-interaction de validation, comme un clignotement ou un cercle qui se remplit) **【48†L100-L109】** **【48†L134-L142】** . Il n'a pas besoin du réseau – tout se stocke localement (rafraîchissement quasi instantané, pas de spinner) **【24†L135-L143】** .
5. **Fin de séance:** résumé de la séance avec totaux. Option de consulter l'historique (liste simple de séances).
6. **Export:** dans le menu, bouton « Exporter mes données ». L'application génère un fichier .json/.csv téléchargeable. Un message rapide affiche « Données exportées  » avec pourcentage de progression si gros.
7. **Parcours desktop/web:**
8. Sur grand écran (PC) l'interface reste minimaliste ; on peut faire un suivi rapide hors salle. Le layout s'adapte (grands boutons, typographie claire). Pas de fonctionnalités spécifiques à desktop, mais tout doit rester utilisable confortablement. Par exemple, le clavier permet d'augmenter les rép (touche espace pour +1).

Chaque parcours est fluide et sans friction grâce aux **micro-interactions** (animations de chargement, changements d'état) qui donnent du feedback immédiat **【48†L100-L109】** . Par exemple, un bouton « Terminer série » peut vibrer/chanter (léger effet visuel) quand on l'atteint.

UI/UX : principes et composants clés

- **Design minimal et premium :** thème clair ou sombre selon réglage OS, formes épurées. Polices grandes et lisibles sur mobile, spacing généreux. Les boutons doivent être suffisamment grands (WCAG touch target ~44px) et centrés.
- **Glassmorphism subtil :** on peut employer des cartes légèrement translucides (verre dépoli) pour les panneaux (par ex. résumé d'exo), mais **sparingly** **【46†L1-L4】** . Le contraste texte/arrière-plan doit rester élevé (normes WCAG) **【45†L183-L192】** . Éviter un fond trop chargé derrière les éléments flous **【45†L183-L192】** **【46†L1-L4】** . Par ex. un card sur fond uni ou légèrement nuancé.

- **Typographie & couleurs** : palette douce et cohérente (ex. neutre + touches colorées de CTA). Texte en “gras modéré” pour titres, normal pour instructions, italique ou plus clair pour détails.
- **Composants clés** :
- **Écran d'accueil / Tableau de bord** : liste des exercices (filtres Today / Tous), icônes simples, bouton d'ajout rapide.
- **Écran exercice en cours** : gros compteur (ex. « Répétitions : 5/12 »), bouton “+1” proéminent, ou un chronomètre, selon exo. Affichage d'une barre de progression ou animation remplie lorsque l'on atteint l'objectif **【48†L100-L109】** . Bouton « Suivant » pour nouvelle série.
- **Historique/Suivi** : liste des séances récentes avec datation et totaux de reps/séries.
- **Écran Export/Paramètres** : choix du format, bouton « Télécharger/exporter », et toggles (p. ex. thème sombre).
- **Composants généraux** : en-têtes, boutons d'action, dialogues modaux (confirmation « Confirmer export ? »), notifications (snackbar en bas « Export terminé ✓ »).
- **Accessibilité** : respecter les normes ARIA et WCAG. Contraste texte/arrière-plan $\geq 4.5:1$. Le texte sur éléments translucides (glass) doit toujours rester lisible **【45†L183-L192】** . Tous les boutons/inputs accessibles au clavier et par VoiceOver.

【33†embed_image】 *Exemple d'interface mobile premium (écran d'accueil illustratif, crédits Dribbble).*

Interaction & animations : Des micro-interactions fluidifient l'usage **【48†L100-L109】** . Par exemple, un clic sur « +1 » est suivi d'une mini-animation (cercle qui pulse, icône qui apparaît) pour confirmer. Un loader circulaire ou barre de progrès traduit la synchronisation hors-ligne/online. Même le passage en mode offline pourrait déclencher un petit bandeau « Vous êtes hors ligne » qui disparaît à la reconnexion.

Architecture technique

- **Stack** : Next.js 15+ avec App Router (TS) + React + Tailwind CSS. Justification : Next.js gère bien le SSR/SEO et PWA (via plugin ou stratégies manuelles **【43†L75-L83】**). Tailwind pour un UI kit rapide et réactif.
- **PWA / Offline-first** : Mettre en place un Service Worker (ex. next-pwa ou le plugin officiel Next.js) pour la cache des assets. Un manifeste web pour installer l'app sur l'écran d'accueil. Stratégie de *cache par défaut* (App Shell) pour l'interface. **Stockage des données via IndexedDB** (ex. bibliothèque Dexie.js ou vue RxDB) **【24†L109-L117】** . Toutes les opérations (ajout de reps, lecture historique) utilisent la base locale. À la première ouverture, le shell et les pages habituelles se chargent de la cache. En cas d'absence de réseau, l'app reste entièrement fonctionnelle (true offline support **【43†L119-L127】**).
- **Synchronisation éventuelle** : Aucune connexion serveur par défaut. Possibilité d'ajouter un mode backup (optionnel) : p.ex. synchroniser vers un service cloud (Google Drive, Supabase) lorsque l'utilisateur est en ligne, sous forme de mise à jour du fichier export. Mais ceci sort du MVP. On se concentre sur un export manuel via fichier.
- **Formats d'export/import** :
- **JSON/CSV** : simples et universels. Ex. fichier JSON contenant tableau d'objets `{exercice, date, series, reps}`. CSV en doublon pour usage tableur.
- **SQLite dump** : optionnel, car sur mobile moins trivial sans moteur local, mais on peut utiliser sql.js pour créer un fichier .sqlite à importer. JSON/CSV suffiront pour V1.
- **Exemple de schéma local** :

```
const schema = {
  exos: {
```

```

    id: 'integer',
    nom: 'string'
  },
  sessions: {
    id: 'integer',
    exoId: 'integer',
    date: 'string',
    series: 'number',
    reps: 'number'
  }
};

```

(Chaque session enregistre l'exercice, date, et totaux de séries/rép.)

- **API (facultative)** : Aucune API externe pour le MVP. Le seul « contrat » réside dans la définition des formats JSON/CSV d'export.
- **Authentification** : Strictement *local only*. On ne gère pas d'utilisateur/logins. Les données sont stockées directement sur l'appareil. Si besoin, on peut protéger l'accès avec un mot de passe/ biométrie local via WebAuthn (hors scope).
- **Déploiement** : Cible principale Vercel (gratuite, optimisée Next.js). Vercel offre le SSL automatique et le CDN pour assets statiques, idéal pour les PWA. Sur Vercel, activer les en-têtes de sécurité (CSP, HSTS, X-Frame-Options) via next.config.js. Un simple `vercel --prod` suffira en CI.

CI/CD et observabilité

- **Pipeline CI minimal** : sur GitHub Actions, déclenché à chaque PR/merge sur `main` :
- **Lint/TypeScript check**: ESLint + `tsc --noEmit`.
- **Tests unitaires** : Jest (ou Vitest) pour la logique métier (sessions, calculs) et React Testing Library pour composants critiques.
- **Tests E2E** : (optionnel) Playwright/Cypress pour un scénario simple (ajout de reps, export).
- **Audit Lighthouse** : via [Lighthouse CI](#) pour noter Performance, Accessibilité, PWA score.
- **Sécurité** : CodeQL analysis (GitHub Actions) + Dependabot automatique. Possibilité d'un scan d'agent ("AgentArmor" [【12†L31-L39】](#)) pour vérifier l'application.
- **Build & déploiement sur Vercel** : si tout passe, déploiement automatique.
- **Observabilité / monitoring** : Peu nécessaire pour une PWA locale, mais on peut intégrer une (optionnelle) librairie comme Sentry pour capturer erreurs JS en production. GitHub Issues sert pour bugs. Les logs sont locaux (console + navigateur).

Sécurité

- **Menaces principales** : étant offline-first, peu de surface réseau, mais on protège contre :
- **Injection XSS/CSRF** : Minimal car pas de serveur, mais les en-têtes CSP/COOP/COEP doivent être activés pour bloquer les scripts tiers (via next.config.js, CSP strict).
- **Fuite de données locales** : les données sont stockées en clair dans IndexedDB. Pour les protéger du vol de l'appareil, on peut suggérer l'option chiffrée (IndexedDB + Web Crypto), mais hors scope MVP. Au moins, ne pas stocker de secrets dans le code.

- **Dépendances vulnérables (supply chain)** : Activer Dependabot (vulnérabilités npm). Utiliser npm audit en CI. Un agent de sécurité IA (p.ex. AgentArmor [12†L31-L39]) peut générer un rapport clair (emojis, % élevés, classification OWASP LLM Top10).
- **Open Redirect/Endpoints** : comme pas d'API, peu de soucis. Désactiver les routes côté serveur non utilisées.
- **Contrôles** :
 - En-têtes de sécurité : CSP avec nonce/sha256 pour scripts, HSTS, X-Frame-Options=SAMEORIGIN.
 - **Content Security Policy** strict pour bloquer toute ressource non approuvée.
 - **Checks RLS**: inutiles en local (pas de base multi-tenant).
 - **CI Security scans** : CodeQL (GitHub Advanced Security) pour code JS/TS, Snyk ou npm-audit pour vulnérabilités.
- **Vérification manuel (agent)** : procédure de type "grille d'audit" agent (comme TailorLayer [13†L247-L256]) pour revue sécurité du projet.

Agent-Driven Development (ADE)

- **Rôles agents (exemples)** :
 - *Agent Produit* : définit le backlog, priorise le MVP scope.
 - *Agent UX/UI* : crée wireframes, guide le design (glassmorphism, accessibilité).
 - *Agent Dev Front* : implémente composants React/Tailwind, Intègre PWA.
 - *Agent Dev Back* (ici stockage) : conçoit le schéma IndexedDB, logique offline.
 - *Agent Sécurité* : gère les scans de vulnérabilités, headers CSP, traitements des dépendances.
 - *Agent QA* : définit et exécute tests, lit rapport Lighthouse.
- **Autonomie et reporting** : Chaque agent travaille en autonomie (pas de micro-management). S'il a un doute, il prend l'initiative de recadrer le besoin ou proposer des alternatives (ex. via la commande fictive "Grill.me" en temps de conception). Chaque fin de sprint/jour ou PR, l'agent produit un **rapport concis** (bullet points ou tableau) avec emojis de statut :
- Exemple de format :
 - ♦ **Avancement**: 80% complété
 - 🟢 **Risque**: faible (5% de bugs potentiels restant)
 - 📋 **Actions**: liste courte (ex. "optimiser la query Dexie").
- On peut utiliser un modèle Markdown pour "Decision Log" :

```
## Décision : Choix du stockage local
- Contexte : on a besoin de stocker persistance client/offline.
- Options : (A) LocalStorage simple, (B) IndexedDB (Dexie), (C) SQLite (sql.js).
- Choix : (B) IndexedDB via Dexie.
- Justifications : support natif, puissant (DB indexée), bien documenté. Pas de burden de dépendance externe (contrairement à sql.js).
- Impact : choix performant pour PWA offline [24†L109-L117] .
```

- **Flux Git / PR** : On utilise la branche principale protégée. Les agents poussent des PR petites et ciblées. Les PR doivent être revus par au moins un autre agent. Le style de commit suit Conventional Commits. Les docs sont maintenues en parallèle (voir section "docs").
- **Outils recommandés** : pull request templates, GitHub Projects pour backlog, GitHub Discussions/Issues pour questions internes. Un outil d'intégration IA (Copilot, DevEnv AI) peut aider à accélérer le dev mais chaque ligne doit être revue selon la politique CI/CD.

Templates et références open-source recommandés

Repo (frontend)	Stars	Licence	Description & usage
creativetim/notus-nextjs	1.8k	MIT	UI Kit Tailwind + pages Next.js. Idéal pour bases visuelles (beau dashboard, composants prêts). Peut être sur-équipé, mais utile pour extraire styles et layouts.
moinulmoin/chadnext	1.3k	MIT	Next.js 15 + ShadCN UI + TypeScript, auth, Stripe. Boîtier full-stack. Adaptable : on peut garder solo front-end et tailwind, ignorer Prisma/Stripe pour MVP.
Skolaczk/next-starter	275	MIT	Starter Next.js + tailwind + tests + NextAuth. Bon template pour architecture propre (CICD, TDD), à alléger (pas besoin Stripe, désactiver NextAuth).
CodeBucks27/dev-portfolio	230	MIT	Exemple simplifié de site perso Next.js+TS+tailwind. Utile pour comprendre flow Next, mais plutôt portfolio. Moins pertinent pour notre usage spécifique d'app.
YashvantHange/AgentArmor	316	MIT	(sécurité agent) Scanner IA de sécurité pour LLM/ chatbots. Bien pour l'approche agentique de sécurité (rapports audit prêts, OWASP LLM). Peut servir de modèle pour intégrer un scan automatisé.
thepeternemec/TailorLayer	165	MIT	Framework JS pour revue de sécurité evidence-driven. Génère rapports de vulnérabilité clairs. Inspirant pour le volet audit CI.
scadastrangelove/awesome-ai-security-tools	1.2k	MIT	Liste collaboratif de tools AI-sécurité (repo). Contient des utilitaires (AgentArmor, etc.). Bon point de départ pour trouver d'autres OSS utiles.

Ces choix couvrent le front-end (UI/templates) et les outils d'audit agentique. Les **templates conseillés** pour forker sont Notus NextJS (pour le style/UI) et ChadNext ou Next-Starter (pour la structure). Les **outils de sécurité open-source** sont AgentArmor et TailorLayer, cités pour leurs rapports « clairs et graphiques » [12†L31-L39] .

Fichiers clés du dépôt (structure minimale)

```

/README.md          # Présentation globale
/PRODUCT.md         # Vision & spé produit (voir plus bas)
/UX_UI.md           # Spécifications UX/UI détaillées
/TECH.md            # Spécifications techniques détaillées
/SECURITY.md        # Politique sécurité, threat model, bonnes pratiques
/ADE.md             # Guidelines Agent-Driven (rôles, décision log)
/next.config.js     # Config Next.js (PWA, Security headers)
/tailwind.config.js # Config Tailwind CSS
/tsconfig.json      # Config TypeScript

```

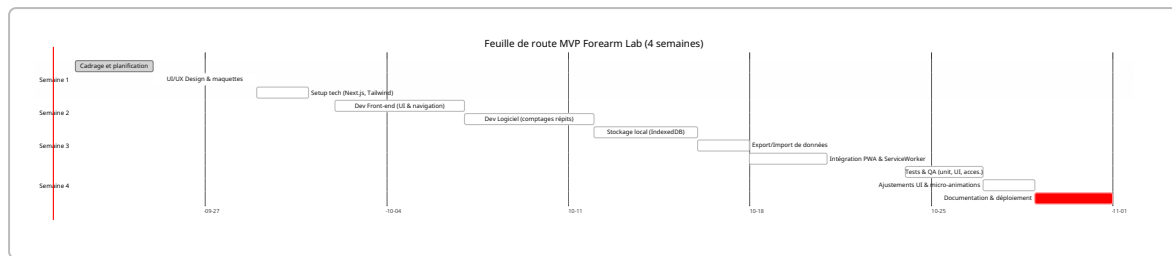
/package.json	# dépendances et scripts
/pages/	# Pages Next.js (app router)
/components/	# Composants UI React (buttons, counters, etc.)
/lib/	# Logique métier (IndexedDB helpers, etc.)
/public/	# Assets (icônes, manifest.json, etc.)
/tests/	# Tests unitaires et e2e
/.github/workflows/	# CI/CD (lint, tests, deploy)

Documentation à produire (5 fichiers)

- **PRODUCT.md** (Cahier des besoins) :
 - Contexte/vision & objectifs
 - Personas, user journeys (mobile & desktop)
 - Fonctionnalités (liste et priorisation) pour MVP et roadmap future
 - Critères d'acceptation / KPIs (facultatif : temps de réponse, score PWA ≥ 90 , SEO)
- **UX_UI.md** (Spécifications design) :
 - Principes de design (mobile-first, glassmorphism modéré **[46†L1-L4]** , micro-interactions **[48†L100-L109]**)
 - Palette de couleurs, typographie, accessibilité (WCAG)
 - Wireframes ou maquettes (écrans clés)
 - Liste des composants réutilisables (ex. Button, Header, Counter, ExportDialog).
 - Règles responsive (layouts pour mobile vs desktop).
- **TECH.md** (Spécifications techniques) :
 - Architecture générale (Stack Next.js+TS, PWA, storage local).
 - Schéma de données (IndexedDB tables, exemple JSON).
 - Workflow "Offline-first" (SW, cache, IndexedDB) avec références **[43†L119-L127]** **[24†L109-L117]** .
 - API (export/import format) contract.
 - Choix d'authent (local-only).
 - CI/CD (build, tests, lighthouse).
 - Diagramme déploiement (Vercel).
- **SECURITY.md** (Guide sécurité) :
 - Menaces & mitigations (ex. injection XSS, vuln. dépendances)
 - Stratégies de défense (CSP, HSTS, RLS, chiffrement)
 - Outils (Dépendabot, CodeQL, AgentArmor/TailorLayer).
 - Politique de revue de code et de vulnérabilités (qui, quand).
- **ADE.md** (Agent-Driven Dev) :
 - Rôles agents et responsabilités (ex. UX, Dev, Sécu)
 - Processus autonome (reporting, questionnement continu)
 - Template de documentation décisionnelle (exemple vu ci-dessus)
 - Format de reporting (emojis, taux de complétion, niveaux de risque).
 - Git/PR rules (commits conventionnels, revue mutuelle).

Chaque doc contiendra une courte introduction suivie de sections en listes/classement. L'idée est d'être opérationnel (checklists, tableaux) plutôt que de longs paragraphes.

Roadmap 4 semaines (MVP)



- **Livrables hebdomadaires** : à la fin de chaque semaine, démonstration du sprint (UI fonctionnel, report de tests).
- **Effort estimé** : ~150 h dev (2 devs agents à mi-temps, plus tests et documentation). Par exemple : S1 (~20h), S2 (40h), S3 (35h), S4 (25h), plus imprévus (30h).

CI Pipeline minimal

- **Build & Lint** : `npm run lint && tsc --noEmit`.
- **Tests** : `npm test` (Jest) + éventuellement `npm run e2e`.
- **Performance/A11Y** : `lighthouse-ci autorun` (report HTML et grade global).
- **Security Scans** : CodeQL Actions (JS/TS), npm audit/Snyk. Optionnel : exécution d'un agent AI (AgentArmor) pour rapport OWASP LLM.
- **Déploiement** : sur `main` validé, déployer sur Vercel (via intégration GitHub-Vercel).

Ainsi, **Forearm Lab** sera robuste, sécurisée, et prête à usage dès le MVP, avec possibilité d'évolutions futures (ajout de synchronisation cloud, plus d'exercices, etc.) [\[43†L119-L127\]](#) [\[48†L100-L109\]](#) .

Sources : Concepts PWA & offline-first [\[43†L119-L127\]](#) [\[24†L109-L117\]](#) , micro-interactions UX [\[48†L100-L109\]](#) , bonnes pratiques glassmorphism [\[46†L1-L4\]](#) [\[45†L233-L241\]](#) , templates Next.js populaires (GitHub topics, rech. automatisées).